

I. Introduction

The *Integrated MFA solution* is an authentication platform designed to provide multi-factor authentication services in business infrastructures with the aim of reinforcing a company's authentication system and protecting its assets against external threats. It is based on popular, actively maintained open-source software and was designed to be compatible with modern enterprise equipment.

When deployed in a business infrastructure, this solution can be integrated with the company's major servers and network equipment, including firewalls, switches, routers...etc. When configured, the solution handles authentication requests and requires users to provide an additional factor of authentication to verify the identity of the user and reduce the risk of compromising data confidentiality, integrity and availability.

This document is the MFA solution's integration guide. It provides step-by-step, clear and detailed instructions for deploying the product from a clean Ubuntu Server installation. It also provides an objective criticism of the current version of the solution, as well as potential perspectives for future improvement. By the end of this document, the reader should have a clear idea of how to setup, configure and deploy the solution.

II. Overview

The solution, at the time of writing, is based on four main components:

1. **FreeRADIUS:** The authentication system
2. **OpenLDAP:** A backend user store
3. **RADIUSdesk:** A web interface for RADIUS server management
4. **TOTP:** An authentication factor, to be integrated with FreeRADIUS

This section gives a brief overview of each component and provides the necessary context for understanding the installation steps provided further down.

FreeRADIUS

FreeRADIUS is an open-source server developed by InkBridge Networks used for network authentication, authorization, and accounting (AAA). It is based on the RADIUS protocol, which outlines communication mechanisms for handling (granting/denying) authentication requests. FreeRADIUS is the core of our MFA solution, and represents the authentication server that ties all the components together in a coherent system.

FreeRADIUS is characterized by a flexible modular design which makes it very powerful. Its main configuration files and directories include:

- **radiusd:** File which stores important configuration parameters
- **mods-available/:** Directory containing a set of modules that extend FreeRADIUS capabilities and compatibility with other protocols or programming languages.
- **mods-enabled/:** Directory containing symbolic links to enabled modules.
- **sites-available/:** Directory containing the *virtual servers* that FreeRADIUS uses. A virtual server is a configuration file describing the instructions and evaluation flow that FreeRADIUS will use to handle authentication requests.
- **sites-enabled/:** Directory containing symbolic links to enabled virtual servers.
- **policy.d/:** Directory containing files that define policies to be used in virtual servers.
- **mods-config/:** Directory containing additional configuration files for modules.

OpenLDAP

OpenLDAP is an open-source server based on the LDAP (Lightweight Directory Access Protocol), which is used for accessing and maintaining distributed directory information services over an IP network. It provides a centralized way to store and manage user, group, and resource information.

In our solution, we use OpenLDAP as a user store to be integrated with FreeRADIUS. During the evaluation of an authentication request, FreeRADIUS queries our OpenLDAP server to check the validity of the provided user credentials and make the decision of granting or denying the request.

OpenLDAP contains a single main configuration file, */etc/ldap/slapd.conf*, which defines the core configuration parameters of the LDAP server.

RADIUSdesk

RADIUSdesk is an open-source, actively maintained web GUI for administering and configuring FreeRADIUS servers. It provides a user-friendly interface for handling different aspects of FreeRADIUS configuration such as managing user accounts, testing authentication, configuring core FreeRADIUS parameters and more.

TOTP

A Time-Based One Time Password is an authentication factor widely adopted in MFA systems. It consists of a temporary, single-usage, six-digit code that refreshes every 30 seconds and that the user must provide in addition to his account credentials to validate an authentication request.

A TOTP is calculated based on two main factors:

- A unique *key* represented by a string. It is critical for the key to be confidential and secure to ensure the strength of the TOTP system.
- Unix time

In our solution, the TOTP must be sent in combination with the password during the authentication request in order for the request to be granted.

III. Integration

FreeRADIUS Installation

As of today, the Ubuntu Server repository contains FreeRADIUS version 3.2.5. We compile by source code to install FreeRADIUS 3.2.7.

```
# Download the FreeRADIUS source code archive from the official website. Replace 3.2.7 by
appropriate version at the time of installation
root@ubs:~# wget https://www.freeradius.org/ftp/pub/freeradius/freeradius-server-3.2.7.tar.gz
# Extract the archive
root@ubs:~# tar -xvzf freeradius-server-3.2.7.tar.gz && cd ./freeradius-server-3.2.7/
```

Once this is done, we find the source code files under the *freeradius-server-3.2.7/* directory, which is named by default after the archive. There are now two methods to proceed with the compilation:

- A method considered classic, consisting of the following three commands:

```
root@ubs:~/freeradius-server-3.2.7# configure
root@ubs:~/freeradius-server-3.2.7# make
root@ubs:~/freeradius-server-3.2.7# make install
```

- An alternative method that uses Ubuntu Server's native package manager, *dpkg*. This procedure is recommended by Dirk Van Der Walt, author of "FreeRADIUS Beginner's Guide", as it allows for a faster and more efficient installation.

We will thus opt for the second method, which we will detail through the following commands:

```
# Modification and update of Ubuntu Server repositories
root@ubs:~/freeradius-server-3.2.7# sed -i 's/Types: deb$/Types: deb deb-src/g'
/etc/apt/sources.list.d/ubuntu.sources
root@ubs:~/freeradius-server-3.2.7# apt update

# Installation of required dependencies
root@ubs:~/freeradius-server-3.2.7# apt install quilt libpcre3-dev libunbound-dev unixodbc-dev
```

```
# Installation of additional tools for package management on Ubuntu Server
root@ubs:~/freeradius-server-3.2.7# apt install dpkg-dev

# Installation of FreeRADIUS dependencies
root@ubs:~/freeradius-server-3.2.7# apt-get build-dep freeradius

# Installation of the fakeroot environment (allows installation with privileges in an isolated and
controlled environment). This package should already be present in the system.
root@ubs:~/freeradius-server-3.2.7# apt install fakeroot

# Installation of the ssl-cert package, required for FreeRADIUS EAP modules
root@ubs:~/freeradius-server-3.2.7# apt install ssl-cert

# Creation of a final control file required for installation
root@ubs:~/freeradius-server-3.2.7# cp ./debian/control.in ./debian/control

# Modify the control file
root@ubs:~/freeradius-server-3.2.7# vi ./debian/control
# The control file contains, at the time of writing, three “@SOMETEXT@” segments. Delete them
as the console does not know how to interpret them and will throw an error otherwise.

# Compilation of the source code in a fakeroot environment. Wait for it to finish
root@ubs:~/freeradius-server-3.2.7# fakeroot dpkg-buildpackage -b -uc

# Upon successful execution of the previous command, the modules of our FreeRADIUS server are
created in the parent directory. These are files ending with the .deb extension. We now install all
the modules necessary for our future implementation
root@ubs:~/freeradius-server-3.2.7# cd ../
root@ubs:~# dpkg -i freeradius_3.2.7+git_amd64.deb freeradius-common_3.2.7+git_all.deb
freeradius-config_3.2.7+git_amd64.deb freeradius-mysql_3.2.7+git_amd64.deb freeradius-
ldap_3.2.7+git_amd64.deb freeradius-python3_3.2.7+git_amd64.deb freeradius-
utils_3.2.7+git_amd64.deb libfreeradius3_3.2.7+git_amd64.deb

# Verify successful installation
root@ubs:~# freeradius -v
```

At this point, FreeRADIUS has been successfully installed. We now proceed to the installation & setup of an LDAP server.

OpenLDAP Installation

To begin, we install our OpenLDAP server from the Ubuntu Server official repository:

```
root@ubs:~# apt install slapd ldap-utils
```

Once done, we proceed to setup and configuration.

First, we adjust OpenLDAP's settings to use the file `/etc/ldap/slapd.conf` as the main configuration file:

```
root@ubs:~# vi /etc/default/slapd
# Here, insert : SLAPD_CONF=/etc/ldap/slapd.conf
```

We then create the `/etc/ldap/slapd.conf` file and insert the following configuration parameters:

```
# Declaring the schemas we will be using
include      /etc/ldap/schema/core.schema
include      /etc/ldap/schema/cosine.schema
include      /etc/ldap/schema/inetorgperson.schema
include      /etc/ldap/schema/nis.schema

# Declaring various configurations parameters
pidfile      /var/run/slapd/slapd.pid
argsfile     /var/run/slapd/slapd.args
modulepath   /usr/lib/ldap
moduleload   back_mdb.la

database     mdb
suffix       "dc=my-domain,dc=com"
checkpoint   1024 5
maxsize      1073741824
rootdn       "cn=Manager,dc=my-domain,dc=com"
rootpw       secret
directory    /var/lib/ldap
```

Once `slapd.conf` is configured, we execute the following commands:

```
# Configure permissions
root@ubs:~# chown openldap: /etc/ldap/slapd.conf
```

```
# Start the slapd service
root@ubs:~# /etc/init.d/slapd start
# Configure launch on startup of the slapd service
root@ubs:~# systemctl enable slapd
```

We now add the FreeRADIUS LDAP schema to our server configuration:

```
# Stop the slapd service
root@ubs:~# /etc/init.d/slapd stop

# We decompress the schema file archive
root@ubs:~# gunzip /usr/share/doc/freeradius/schemas/ldap/openldap/freeradius.schema.gz

# We copy the file to the OpenLDAP schema directory
root@ubs:~# cp /usr/share/doc/freeradius/schemas/ldap/openldap/freeradius.schema
/etc/ldap/schema/freeradius.schema

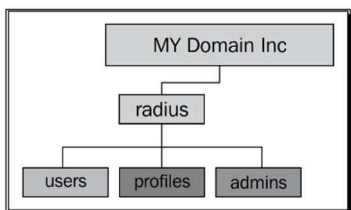
# We include the schema in slapd.conf
root@ubs:~# vi /etc/ldap/slapd.conf
# Here, we add the declaration: "include /etc/ldap/schema/freeradius.schema" in the file

# We restart the slapd service
root@ubs:~# /etc/init.d/slapd restart
```

At this point, we are done with the installation and setup of the LDAP server. It is now time to populate it with user accounts. In a production scenario, this would be done by importing user accounts from an existing LDAP server, such as the FreeIPA server that Tusna & data uses. However, as this machine is used for testing purposes, we will simply use an existing LDIF file copied from the “FreeRADIUS Beginner’s Guide”, page 107, along with some minor modifications due to versioning differences. The final LDIF file will be included in the VM as `/home/admin1/ldap_user_store.ldif`

We now proceed to add the user accounts to our LDAP server:

```
root@ubs:~# ldapadd -x -D 'cn=Manager,dc=my-domain,dc=com' -w secret -f ldap_user_store.ldif
```



The figure to the left illustrates the resulting Directory Information Tree of our LDAP server. Under the *users* OU, we find our various user accounts, namely *student1*, *student2* and *student3*.

Note: Each account's password is identical to its username.

Our LDAP server is now populated with user accounts and is ready to be used for testing purposes.

Our next step will be to configure the LDAP module of FreeRADIUS to ensure a smooth integration between the two servers. We will find the relevant configuration file under the directory `/etc/freeradius/mods-available/`

Note: In the next step, we will add a web UI to manage our FreeRADIUS server. The web interface will use its own FreeRADIUS configuration, which will be managed through a GUI rather than the command line. Due to this, some of the following configuration steps will become obsolete as soon as the web interface is implemented. This being said, they are still included in this document for the sake of completeness.

We begin our configuration of the LDAP module:

```
root@ubs:~# vi /etc/freeradius/mods-available/ldap
# We add the following entries within the "ldap { }" section of the file:
server = '127.0.0.1' # LDAP server address
identity = 'cn=Manager, dc=my-domain, dc=com' # Administrator with privileges on the
# LDAP server
password = 'secret' # Administrator password
base_dn = 'ou=users, ou=radius, dc=my-domain, dc=com' # DN of the parent OU of all user entries
# We add the following entry under the "user { }" subsection of the file. Be sure to comment out
the existing filter.
filter = '(cn=%{%{Stripped-User-Name}-%{User-Name}})' # Processing syntax during entry searches
```

The FreeRADIUS LDAP module is now configured successfully. All that's left is to include it within the default virtual server's processing flow and test our configuration:

```
# We modify the virtual server
root@ubs:~# vi /etc/freeradius/sites-enabled/default
# We add the following line under the "authorize { }" section if it is not already present
ldap
# We add the following lines under the "authenticate { }" section
Auth-Type LDAP {
    ldap
}
```



```
# We enable the LDAP module by creating a symbolic link
root@ubs:~# ln -s /etc/freeradius/mods-available/ldap /etc/freeradius/mods-enabled/ldap

# We restart the FreeRADIUS server
root@ubs:~# systemctl restart freeradius

# We test our configuration and ensure that it was successful
root@ubs:~# radtest student1 student1 127.0.0.1 100 testing123
```

At this point, if the *radtest* command outputted an *Access-Accept*, then we have finished our LDAP module configuration.

RADIUSdesk Implementation (Web UI)

To ensure administrative management of FreeRADIUS, we proceed to install a web-based solution designed for this purpose. In this document, we choose to implement the popular open-source platform for managing FreeRADIUS: **RADIUSdesk**

RADIUSdesk's installation is clearly documented in great detail in the product's website. As such, the step-by-step instructions will not be included in this document. To proceed with the web interface's installation, follow the instructions included in the following link: https://radiusdesk.com/wiki24/install_24_4

After following the instructions in the installation page of the RADIUSdesk wiki, we proceed with a few additional steps to complete RADIUSdesk's setup:

```
# We install a few more dependencies needed for RD's integration with FreeRADIUS
root@ubs:~# apt-get -y install libdatetime-perl libdbd-mysql-perl libdigest-hmac-perl libdatetime-
format-rfc3339-perl eapoltest

# Stop the service if it is already running
root@ubs:~# systemctl stop freeradius

# Backup the original FreeRADIUS directory
root@ubs:~# mv /etc/freeradius /etc/freeradius.orig

# Extract the RADIUSdesk modified FreeRADIUS directory
root@ubs:~# tar xzf /var/www/html/cake4/rd_cake/setup/radius/freeradius-radiusdesk.tar.gz --
directory /etc

# Configure appropriate permissions
root@ubs:~# chown -R freerad: /etc/freeradius/3.0/
root@ubs:~# mkdir /var/run/freeradius
root@ubs:~# chown freerad: /var/run/freeradius

# At this stage, we need to make adjustments to the FreeRADIUS service due to versioning
# differences. The FreeRADIUS service we originally installed stores its configuration files within the
# /etc/freeradius/ directory. However, the RADIUSdesk pre-configured version stores its files in the
# /etc/freeradius/3.0 directory. This difference must be accounted for before FreeRADIUS can
# launch successfully.

# We adjust the service file
root@ubs:~# vi /lib/systemd/system/freeradius.service
```

```
# Here, we navigate to lines 36, 37 and 40, and add the argument "-d /etc/freeradius/3.0" at the
# end of each line
ExecStartPre=/usr/sbin/freeradius ... -d /etc/freeradius/3.0
ExecStart=/usr/sbin/freeradius ... -d /etc/freeradius/3.0
...
ExecReload=/usr/sbin/freeradius ... -d /etc/freeradius/3.0

# We exit the file and reload the systemd daemon
root@ubs:~# systemctl daemon-reload

# Note: If launching FreeRADIUS in debug mode (freeradius -X), be sure to add the above
parameter as well (-d /etc/freeradius/3.0), or else the command will fail

# Restart FreeRADIUS and check successful launch
root@ubs:~# systemctl restart freeradius
root@ubs:~# systemctl status freeradius
```

At this point, the webpage is operational and we can load it on our browser by entering the IP address of the VM. An authentication portal is displayed, and we can log in using the default credentials:

- **Username:** root
- **Password:** admin *(The default password may be changed later on)*

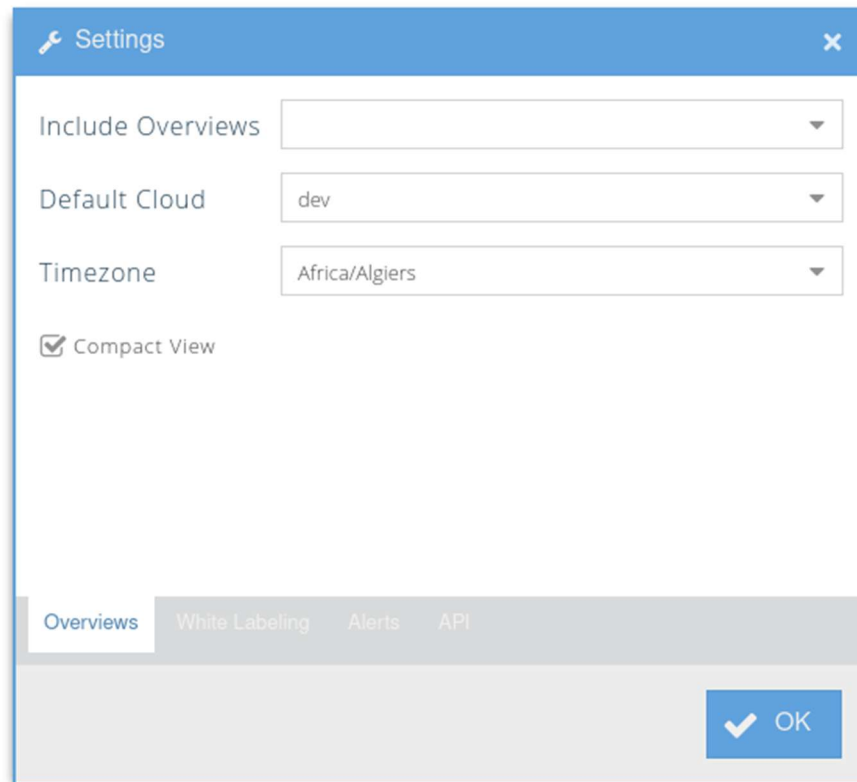
A setup wizard then displays, prompting us to create our first “Cloud” within RADIUSdesk. A *cloud* is somewhat equivalent to the concept of *realms* within FreeRADIUS. However, more details are available in the RADIUSdesk wiki for an exact definition.

At this stage, we do not create our cloud yet. In accordance with the wiki page https://radiusdesk.com/wiki24/install_tweak, we first close the setup wizard window and navigate to **Other (Gear icon on the left)-> Settings**. We proceed to adjust the following parameters: *Password, Country, Timezone & RADIUS-1*. Once this is done, we reload the page, re-authenticate and follow the steps of the setup wizard to create our first cloud.

Next, we configure the overview screen by *clicking on the logged-in user drop-down arrow at the top right of the screen and navigating to settings*. A small window opens at the center of the screen as can be seen below.

We begin by adjusting the timezone the relevant value. Then, in the “Include Overviews” section dropdown, we first add “Networks”, select our previously created cloud in “Default Cloud”, and click OK. We reload the page, and the change takes effect.

Next, we repeat the same procedure to add the “*Data Usage (RADIUS)*” overview screen, with the additional step of selecting the “*** ALL REALMS ***” value in the “*Default Realm*” section.



Note: Regarding the above two steps, the wiki page instructs us to add both “*Networks*” and “*Default Cloud*” at the same time. However, at the time of writing, RADIUSdesk has a bug that does not allow for this to be done. If both overview screens are added at the same time, the “*Default Realm*” drop-down will be empty and it will not be possible to complete the procedure as it is a mandatory field and its value cannot be left blank. The only workaround to this bug is to add the two overview screens separately.

In the “*White Labeling*” tab at the bottom of the window, options are given to adjust or modify interface elements including *header foreground/background color*, *header/footer text & website logo*. This allows for production implementations of our solution to be customized according to client preferences.

Warning: The image file used for the website logo must not be more than 48 px in height. Otherwise, the website interface may become partly warped and it will become impossible to reverse the modification or access user parameters.

In this testing implementation, we have chosen to insert the Tusna & data logo, and modified both the header and footer text to “TndDesk”.

Enabling RADIUS LDAP Integration

By default, RADIUSdesk does not configure its RADIUS server to use the machine’s LDAP server as a user store for authentication. Therefore, we have to enable this functionality ourselves.

We begin by configuring the *ldap* module in */etc/freeradius/3.0/mods-available*:

```
root@ubs:~# vi /etc/freeradius/3.0/mods-available/ldap
# Here, we configure the necessary parameters for the RADIUS server to be able to bind to our
# LDAP server
# Inside the ldap {...} section, insert the following:
server = '127.0.0.1'                                # LDAP server's IP address
identity = 'cn=Manager,dc=my-domain,dc=com'         # Root account DN
password = 'secret'                                  # Root account password
base_dn = 'ou=users,ou=radius,dc=my-domain,dc=com'  # Base DN of user acc searches

# Inside the user {...} subsection, insert the following:
filter = "(cn=%{Stripped-User-Name}:-%{User-Name})"
# Exit and save the file

# Create a symlink to the ldap module in the mods-enabled/ directory
root@ubs:~# cd /etc/freeradius/3.0/mods-enabled
root@ubs:/etc/freeradius/3.0/mods-enabled# ln -s ../mods-available/ldap ldap

# Enable LDAP authentication in the default virtual server
root@ubs:~# vi /etc/freeradius/3.0/sites-enabled/default
# Inside the authenticate {...} section, uncomment the LDAP subsection
Auth-Type LDAP {
    ldap
}

# Restart the RADIUS server
root@ubs:~# systemctl restart freeradius
```

At this point, the RADIUS LDAP integration has been fully configured and our RADIUS server is now able to use LDAP as a backend for checking user credentials.

However, in order for present and future testing to work properly, it is important to understand how RD’s RADIUS server handles authentication requests.

By default, RADIUSdesk has its virtual server configured to use a MariaDB database called 'rd' as backend for storing user credentials. RD stores user accounts within a table called *radcheck*, which we can see an example of below:

```
MariaDB [rd]> SELECT * FROM radcheck;
```

id	username	attribute	op	value
10623	dev@dev	User-Profile	:=	dev
10624	dev@dev	Rd-Realm	:=	dev
10625	dev@dev	Rd-Account-Disabled	:=	0
10626	dev@dev	Cleartext-Password	:=	Tusna123!
10627	dev@dev	Rd-User-Type	:=	user
10628	click_to_connect@dev	User-Profile	:=	dev_Click-To-Connect
10629	click_to_connect@dev	Rd-Realm	:=	dev
10630	click_to_connect@dev	Rd-Account-Disabled	:=	0
10631	click_to_connect@dev	Cleartext-Password	:=	click_to_connect
10632	click_to_connect@dev	Rd-User-Type	:=	user

```
10 rows in set (0.004 sec)
```

The benefit to this approach is centralized and simplified management of user accounts. However, RADIUSdesk is flexible and allows for handling authentication requests using other backend sources such as LDAP. The way this is done is via the use of a custom private attribute called '*Rd-Auth-Type*'.

When an authentication request is sent to RD's RADIUS server, it starts by searching for the requested username within the *radcheck* table. If a matching entry is found, its attributes are evaluated one by one. If the account's '*Rd-Auth-Type*' attribute is set, then its credentials will be evaluated by whatever method *Rd-Auth-Type*'s value was set to.

In other words, in order to use LDAP as a user store during RADIUS authentication, user accounts must have an entry inside the *radcheck* table which associates their username with the *Rd-Auth-Type* attribute and sets its value to '*LDAP*'.

With this information in mind, we can now test the successful implementation of our LDAP integration:

```
# We first insert an entry inside the radcheck table containing the student1 username, which we
# created inside our LDAP server earlier
root@ubs:~# mysql -u rd -p                                     # Enter 'rd' as password
MariaDB [(none)]> USE rd;                                       # Use the 'rd' database
MariaDB [(rd)]> INSERT INTO radcheck(username, attribute, op, value) VALUES('student1', 'Rd-
Auth-Type', ':=', 'LDAP');                                     # Insert our entry
MariaDB[(rd)]> quit;
```

```
# Test successful implementation
```

```
root@ubs:~# radtest student1 student1 127.0.0.1 100 testing123
```

TOTP Implementation

At this point, we have arrived at the last step in the integration of our MFA project. The implementation of a TOTP system within a FreeRADIUS-based solution is a process that can be easily configured thanks to FreeRADIUS' flexibility and modular design.

At a point in FreeRADIUS' development, an OTP module designed to handle one-time password use cases was developed and integrated in subsequent FreeRADIUS releases. However, this module is now officially considered deprecated and should not be used (see: <https://networkradius.com/doc/current/raddb/mods-available/otp.html>).

Currently, the recommended way to integrate OTP functionality within FreeRADIUS is to make use of FreeRADIUS' support for simple scripting/programming languages such as *Perl* or *Python*. In this integration document, we will be describing how to configure this feature using Python.

Enabling and configuring the Python module

While FreeRADIUS natively includes a Python module inside its *mods-available/* directory, it is neither enabled nor configured by default. Thus, our first step will be to accomplish both of those tasks.

We begin by making the necessary changes to the *python* module:

```
root@ubs:~# vi /etc/freeradius/3.0/mods-available/python
# In this file, declarations are included for every single virtual server section (here called functions)
# which may call the python module. It is recommended to comment out all functions that WILL
# NOT be calling the python module, and leave only those that will – in our case, authenticate.

python3 python{
    python_path = /etc/freeradius/3.0/mods-config/python
    module = example

    mod_authenticate = ${module}
    func_authenticate = authenticate
}
# The rest of the code in this file should be commented, as it will otherwise cause the module to
# fail on startup.
# In this configuration, we have defined a python script, namely /etc/freeradius/3.0/mods-
# config/python/example.py, that will be executed when the python module is called from the
# virtual server. More specifically, a different function inside example.py will be executed
```



```
# depending on which virtual server section called the python module (for example, if the
# authenticate {...} section called the python module, then the authenticate function of
# example.py will be executed)

# We replace the default example.py module included with the RADIUSdesk configuration with the
# one we installed at the beginning of this integration process
root@ubs:~# cd /etc/freeradius/3.0/mods-config/
root@ubs:~# mv python python.backup
root@ubs:~# cp -r /etc/freeradius.orig/mods-config/python3 python

# Restart the FreeRADIUS server
root@ubs:~# systemctl restart freeradius
```

At this point, the python module will initiate properly on startup, and its debug logs can be observed in the *authorize {...}* section during authentication requests if FreeRADIUS is running in debug mode.

We have now successfully configured and enabled Python functionality within FreeRADIUS. All that's left is to write the Python script necessary to implement TOTP mechanisms.

Before we begin writing the script, we first need to install the Python packages we will be using:

```
# Install necessary packages
root@ubs:~# apt install python3-pyotp python3-qrcode python3-ldap3
```

Writing the script:

```
root@ubs:~# vi /etc/freeradius/3.0/mods-config/python/example.py
# Here, we begin to implement our script
# First thing we need is to import the needed packages
import radiusd
import time
import pyotp
import qrcode
import ldap3
import base64
# Define an authenticate function right under the authorize function
# The p argument is a "tuple of tuples" passed to the Python module when it is called by the
# virtual server. It contains the AVPs of the authentication request as tuples.
def authenticate(p)
    # The following lines may be omitted. They are used for debugging only and are not part of
    # the TOTP functionality.
```

```

print("*** authenticate ***")
radiusd.radlog(radiusd.L_INFO, '*** radlog call in authenticate ***')
print()
print(p)
print()
print(radiusd.config)
# Here, our script begins
# First step is to retrieve the username & password included in the request
for key, value in p:
    if key == 'User-Name':
        userid = value.strip(' "') # Strip the username of quotation marks
        print("\nuserid=",userid) # Debug lign
    if key == 'User-Password':
        otpcode = value[-6:].strip(' "') # Retrieve OTP and format
        userpw = value[:-6].strip(' "') # Retrieve password and format
        print("\nuserpw=",userpw,"\notpcode=",otpcode)
# At this point, we have retrieved the username included in the request, the OTP code
# (last six characters), as well as the password (the whole string up to the last six
# characters). Additionally, we've done necessary formatting by stripping the values of
# quotation marks.

# The next step is now to generate the QR code which the user will scan with an
# authentication app in order to retrieve the TOTP needed for his login
# First, we generate a unique key, which we will need later to generate a TOTP. This key is
# based on a combination between the account's username and a random string.
# Note: The key is the core behind the uniqueness and the secrecy of the TOTP, and for
# that reason it should remain STRICTLY CONFIDENTIAL. Otherwise, the TOTP becomes
# easily replicable and loses its value as a security asset.
raw_key = userid + "topsecretkey"
# Next, we encode the key into the necessary format for the TOTP algorithm
key = base64.b32encode(raw_key.encode()).decode()
# We calculate the TOTP based on the key
totp = pyotp.TOTP(key)
# We create a URI which will be used for sharing the TOTP with the user
uri = pyotp.totp.TOTP(key).provisioning_uri(name="ro8",issuer_name="Tnd")
# We save the URI as a QR code for easier sharing
qrcode.make(uri).save("/tmp/totp.png")

```

Note: It is important to keep in mind that this Python script's primary objective is to evaluate a request based on the included TOTP, and deny or validate the request based on the result of the evaluation. At this point in the script, we have just finished writing the code for sharing the TOTP with the user as a QR code. This means that we now face the following paradox:

- On one hand, the user must include the correct TOTP in order to be able to log in.

- On the other hand, the user must first *attempt* to log in before he can have the QR code needed to obtain the correct TOTP.

This means that the first request of the user is always destined to fail, and represents merely a formality. After a failed first authentication request, the user can access the QR code at `/tmp/totp.png`, scan it, and include the correct TOTP in subsequent authentication attempts.

In this first version of the MFA project, the script was written in this way with the aim of saving time.

However, in future versions, it is recommended to generate and share the QR code through a separate script, which should run without the need for a failed authentication request. It is also recommended to include an automated and effortless way of sharing the script with the concerned user (for example, via email).

We continue the writing of our script:

```
# We setup the necessary parameters for an LDAP server bind
LDAP_SERVER = 'ldap://127.0.0.1'
BASE_DN = 'ou=users,ou=radius,dc=my-domain,dc=com'
USERNAME = userid
PASSWORD = userpw
USER_DN = f'cn={USERNAME},{BASE_DN}'
# We attempt a bind using the provided credentials and parameters
conn = ldap3.Connection(LDAP_SERVER, user=USER_DN, password=PASSWORD)
# If the authentication succeeds, then the provided username/password combination is
# correct
if conn.bind():
    print("\nAuthentication successful")
    conn.unbind()
# If it fails, then the combination is incorrect and the authentication request is
# automatically rejected
else:
    print("\nAuthentication failed")
    # Provide a small explanation
    update_dict = {
        "request": (),
        "reply": (("Reply-Message", "Invalid username or password"),),
        "config": (),
    }
    return radiusd.RLM_MODULE_REJECT, update_dict
# At this point, if the script is still running, then that means the authentication succeeded,
# and we can move on to OTP verification.
```

```

# We perform a simple comparison
if otpcode == str(totp.now()):
    update_dict = {
        "request": (),
        "reply": (("Reply-Message", "OTP confirmed. Welcome!"),),
        "config": (),
    }
    return radiusd.RLM_MODULE_OK, update_dict
else:
    update_dict = {
        "request": (),
        "reply": (("Reply-Message", "Invalid OTP provided. Try again"),),
        "config": (),
    }
    return radiusd.RLM_MODULE_REJECT, update_dict

```

At this stage, we have finished the writing of our script.

However, one last step remains. While the TOTP logic is fully completed, we still need to include this Python module inside of FreeRADIUS' evaluation flow for authentication requests.

To do this, we must edit the default virtual server that will handle our requests:

```

root@ubs:~# vi /etc/freeradius/3.0/sites-enabled/default
# At the end of the authorize {...} section, insert the following:
    update control {
        Auth-Type := OTP
    }
# This updates the Auth-Type AVP of the request and assigns it the value "OTP"
# Now, inside the authenticate {...} section, insert the following:
    Auth-Type OTP {
        python
    }

```

In this last step, we did the following:

- At the end of the *authorize {...}* section of the default virtual server, we systematically update the Auth-Type AVP of the request, and set it to our own custom value, in this case "OTP". The reason this is done at the end of the section is to prevent the AVP from being modified by subsequent lines of code.
- Inside the *authenticate {...}* section, we define the logic for evaluating an authentication request with *Auth-Type = OTP*. By calling the *python* module inside this section, we execute the *authenticate* function of the *mods-config/python/example.py* file, which contains all the logic behind our TOTP functionality. This function then handles the authentication and takes charge of granting/denying access.

It is now time to move on to testing our solution:

```
# Launch a first authentication request to generate the QR code
root@ubs:~# radtest student1 randompassword 127.0.0.1 100 testing123
# An Access-Reject is returned. We verify that the QR code was successfully generated
root@ubs:~# ls /tmp/totp.png
# A result is returned, confirming that the QR was successfully generated. The next step is now to
# scan it.
```

Since we cannot open an image file on an Ubuntu Server VM, we first transfer it to our Windows machine.

```
C:\Users\Ro8> scp admin1@10.100.10.42:/tmp/totp.png C:\Users\Ro8\Desktop\totp.png
```

We open the QR code and scan it with an authentication app such as Microsoft Authenticator. After doing this, we now have access to the TOTP associated with our *student1* account. We can thus retry a second authentication attempt, which will be met with an *Access-Accept*.

```
root@ubs:~# radtest student1 student1069419 127.0.0.1 100 testing123
```

At this stage, our integration can officially be considered *complete*.